5-1 · 분산 처리와 Spark 아키텍처 · 실습 1.5H

단일 노드 Spark 기동과 PySpark 첫 실행

PySpark 를 띄우고 지연 평가를 눈으로 확인

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① apache/spark:3.5.3 컨테이너로 PySpark 셸을 띄운다

② transformation 만 걸면 Completed Jobs 가 없음을 본다

③ count() 는 Job 2개, show(5) 는 Job 3개 — 직접 센다

④ explain() 을 아래에서 위로 읽고 파티션 수를 확인한다

여기서 본 파티션 수가 다음 1H 실습에서 Task 수로 다시 나타납니다

실습 흐름

일곱 단계 — 셸 하나에서 계획·실행·계획서까지

python 상자는 >>> 에 붙여 넣고, bash 상자는 새 터미널 창에서 칩니다

1. 셸 기동	2. 환경 확인	3·4. 계획→실행	5. 실행 계획	6·7. 파티션·과제
docker run -p 4040:4040	version master	transformation action	explain() 일부러 실패	getNumPartitions 미니 과제

1·2단계

기동과 환경 확인 — 명령 다섯 줄

기동	docker run -it --rm -p 4040:4040 apache/spark:3.5.3
셸	/opt/spark/bin/pyspark (run 명령 끝에 이어서)
버전	spark.version
병렬도	spark.sparkContext.defaultParallelism
마스터	spark.sparkContext.master

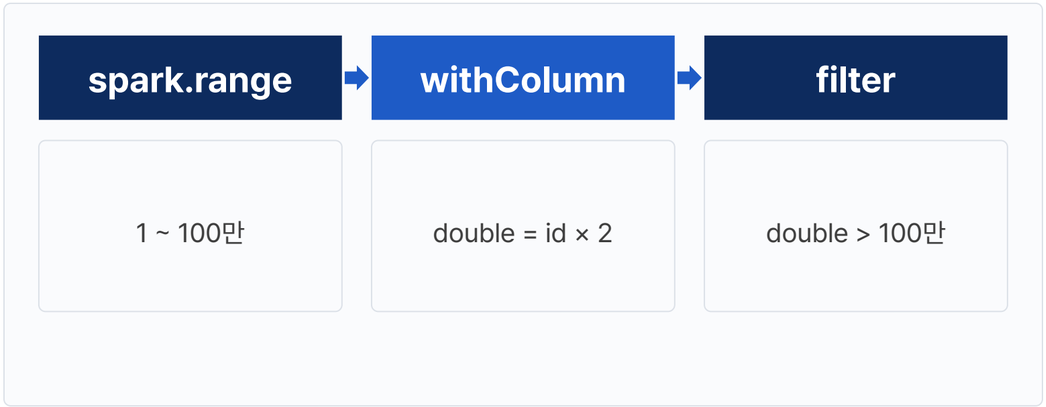
확인: 결과 '3.5.3' · 16 · 'local[*]' — 가운데 숫자는 Docker 에 준 CPU 수라 사람마다 다름

3·4단계

transformation 은 즉시 반환, action 은 실행

3단계 — transformation 만

즉시 반환 · 4040 은 열리지만 Completed Jobs 없음



VS

4단계 — action

action 하나가 Job 여럿을 만들기도 한다



5단계

explain() 은 아래에서 위로 읽는다

Physical Plan	읽는 순서	뜻
***(1) Project**	③ 마지막	double 계산
+- *(1) Filter	② 두 번째	id * 2 > 1000000
+- *(1) Range	① 처음	splits=16
explain(True)	Parsed → Optimized	Filter 가 Project 아래로 내려감

코드는 withColumn 을 먼저 썼지만 계획은 조건(id * 2 > 1000000)을 계산 앞으로 당겼다 — 걸러진 행은 double 을 계산할 필요가 없다

5단계 — 일부러 실패시키기

다른 DataFrame 의 컬럼을 끌어다 쓴다

쓰지 않을 컬럼 x 를 만들려다 흔히 하는 실수 — action 전에 바로 막힙니다

잘못 쓴 줄	withColumn 에서	분석 단계 오류	F.lit(1) 로	q.explain()
spark.range(1).id * 0 + 1	Traceback explain 전	AnalysisException id 를 못 찾음	상수 컬럼은 lit 으로	Filter · Range 두 줄뿐

6단계

파티션 수를 본다 — 이 수가 곧 Task 수

기본 파티션 수	df.rdd.getNumPartitions()
필터 뒤	filtered.rdd.getNumPartitions()
import	from pyspark.sql.functions import spark_partition_id
파티션별 행 수	df.groupBy(spark_partition_id()).count().show()

확인: 두 숫자가 defaultParallelism 과 같고(실측 16 · 16) 파티션 0~15 가 62500 씩 고르게(100만 ÷ 16) — 다음 1H 에서 UI 로 확인

7단계

미니 과제 세 가지

1. **1,000만 행에서 짝수 세기**
   spark.range(1, 10_000_001) 로 만들고 짝수만 count — 시간이 얼마나 걸리는지

2. **filter 두 번 이어 걸기**
   explain() 으로 두 조건이 하나로 합쳐졌는지 본다

3. **transformation 5개, action 없이**
   UI 를 새로고침해 새 Job 이 생기지 않는지 본다

count() 와 show(5) 가 따로 Job 을 만든다 = action 마다 처음부터 다시 계산 — cache() 는 다음 블록 6단계에서 재 봅니다

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. >>> 프롬프트와 「SparkSession available as spark」가 떴다
2. '3.5.3' · 숫자 하나(실측 16) · 'local[*]'
3. transformation 뒤 Completed Jobs 가 없다
4. count() 500000 · Job 2개, show(5) 로 Job 3개 더
5. explain() 을 아래에서 위로 — Range → Filter → Project
6. 파티션 16 · 0~15 가 62500 씩(코어 수에 따라 다름)

5-1 · 실습 1.5H 마무리

다음은 Spark UI로 잡 구조 읽기 (1H)

지금 띄운 셸을 그대로 두고 localhost:4040 에서 Job → Stage → Task 와 DAG 를 읽습니다

- ✔ **Jobs 탭 — count · showString 다섯 줄**
- ✔ **groupBy 셔플로 Stage 가 갈린다**
- ✔ **Stages 탭의 Task · DAG · Executors**
- ✔ **cache 뒤 Storage 탭 · 셸 하나 더(실패)**

가져갈 것

셸을 끄지 말 것 · 파티션 수 메모